

TEORIA DA COMPUTAÇÃO

Recursão primitiva de Kleene

Construções passo a passo, exemplos numéricos e equivalência com Máquinas de Turing

Objetivo do material: compreender a notação da recursão primitiva, construir funções aritméticas sobre os naturais e entender por que as funções parciais recursivas de Kleene têm o mesmo poder computacional das Máquinas de Turing.

Material didático consolidado

Como ler este material

O texto está dividido em duas partes. A primeira apresenta a recursão primitiva exatamente na notação escolhida e constrói funções conhecidas. A segunda mostra como a minimização não limitada amplia o modelo e conduz às funções parciais recursivas de Kleene, equivalentes às Máquinas de Turing.

Parte	Questão central	Resultado esperado
I	Como definir funções por recursão primitiva?	Soma, subtração, multiplicação, divisão, fatorial e médias.
II	Por que Kleene fornece um modelo universal?	Equivalência entre funções parciais recursivas e Máquinas de Turing.

Convenção: adotaremos $\mathbb{N} = \{0, 1, 2, 3, \dots\}$. Portanto, todas as funções básicas recebem naturais e devolvem naturais. Quando quisermos representar notas decimais, usaremos uma codificação por escala.

Parte I - Recursão primitiva

1. Ideia fundamental

Uma definição recursiva descreve o valor de uma função usando valores anteriores da própria função. Na recursão primitiva, essa repetição é controlada por um contador natural que diminui até chegar a zero. Por isso, a computação sempre tem um número finito de etapas.

Pense em uma escada. Para chegar ao degrau y , começamos no degrau 0 e aplicamos a regra de passagem exatamente y vezes. A regra não escolhe livremente quanto tempo continuar: o próprio argumento y determina a quantidade de passos.

Ponto-chave: na recursão primitiva há um caso-base e um caso recursivo. O caso recursivo sempre usa o predecessor do contador. Assim, y , $p(y)$, $p(p(y))$, ... inevitavelmente chega a 0.

2. Funções e símbolos básicos

Símbolo	Significado	Exemplo
$z(x) = 0$	Função zero: devolve 0 para qualquer entrada.	$z(17) = 0$
$s(x) = x + 1$	Sucessor: soma uma unidade.	$s(4) = 5$
$p(0) = 0; p(x+1) = x$	Predecessor truncado: retira uma unidade sem produzir número negativo.	$p(4) = 3; p(0) = 0$
π_i^n	Projeção: escolhe o i-ésimo argumento entre n argumentos.	$\pi_2^3(7, 9, 4) = 9$
$\text{cond}(C, A, B)$	Condicional: devolve A quando C é verdadeira e B caso contrário.	$\text{cond}(3=0, 8, 5) = 5$

Atenção à projeção: π_i^n não guarda histórico e não executa repetição. Ela apenas seleciona um argumento. Na definição formal de uma função, a projeção é usada para entregar à regra exatamente o parâmetro de que ela precisa.

3. A notação escolhida

Considere uma função f de $n+1$ argumentos. Os primeiros n argumentos, $x_1, \dots, x_n$, permanecem disponíveis durante toda a computação. O último argumento, y , é o contador sobre o qual ocorre a recursão.

$$f(x_1, \dots, x_n, y) = \text{cond}(y = 0, g(x_1, \dots, x_n), \\ h(x_1, \dots, x_n, p(y), f(x_1, \dots, x_n, p(y))))$$

A mesma definição, escrita por casos, é:

$$f(x_1, \dots, x_n, 0) = g(x_1, \dots, x_n) \\ f(x_1, \dots, x_n, y) = h(x_1, \dots, x_n, p(y), f(x_1, \dots, x_n, p(y))) \quad \text{se } y \neq 0$$

Elemento	Papel na definição
f	É a nova função que está sendo definida.
y	É o contador recursivo. Ele decide quando usar o caso-base.
g	Calcula o valor inicial quando $y = 0$.
h	Calcula o próximo valor a partir dos parâmetros, do índice anterior $p(y)$ e do resultado recursivo anterior.
$f(\dots, p(y))$	É o resultado já calculado no passo anterior.

Leitura operacional: se $y = 0$, pare e use g. Se $y \neq 0$, primeiro calcule f nos dados com contador $p(y)$; depois entregue esse resultado à função h para construir o próximo valor.

4. Soma

Para calcular $\text{soma}(x,y)$, mantemos x fixo e usamos y como contador. Somar zero não altera x. Cada unidade presente em y provoca uma aplicação do sucessor.

$$\begin{aligned}\text{soma}(x, y) &= \text{cond}(y = 0, x, s(\text{soma}(x, p(y)))) \\ \text{soma}(x, 0) &= x \\ \text{soma}(x, y) &= s(\text{soma}(x, p(y))) \quad \text{se } y \neq 0\end{aligned}$$

Na forma geral, $g(x) = x$ e $h(x, i, r) = s(r)$. O argumento i representa o índice anterior, mas a soma não precisa usá-lo. O símbolo r representa o resultado anterior.

Etapa	Expansão de $\text{soma}(3,2)$	Resultado disponível
1	$\text{soma}(3,2) = s(\text{soma}(3,1))$	Ainda depende de $\text{soma}(3,1)$
2	$\text{soma}(3,1) = s(\text{soma}(3,0))$	Ainda depende do caso-base
3	$\text{soma}(3,0) = 3$	Caso-base: 3
4	$\text{soma}(3,1) = s(3)$	4
5	$\text{soma}(3,2) = s(4)$	5

Intuição: $\text{soma}(3,2)$ significa começar em 3 e aplicar o sucessor 2 vezes: $3 \rightarrow 4 \rightarrow 5$.

5. Subtração truncada

Como estamos trabalhando apenas com naturais, a subtração não pode devolver um número negativo. Usamos a subtração truncada, também chamada de monus: quando y é maior que x , o resultado é 0.

$$\text{sub}(x, y) = \text{cond}(y = 0, x, p(\text{sub}(x, p(y))))$$

$$\text{sub}(x, 0) = x$$

$$\text{sub}(x, y) = p(\text{sub}(x, p(y))) \text{ se } y \neq 0$$

Cálculo	Desenvolvimento	Resultado
$\text{sub}(5, 3)$	$p(p(p(\text{sub}(5, 0)))) = p(p(p(5)))$	2
$\text{sub}(2, 5)$	$p(p(p(p(p(2))))))$	0, pois $p(0)=0$

Importante: $\text{sub}(2, 5)=0$ não significa que $2-5$ seja 0 na aritmética dos inteiros. Significa apenas que a função foi definida em $\mathbb{N}$ e, portanto, foi truncada em zero.

6. Multiplicação construída com a soma

Multiplicar x por y equivale a somar x a si mesmo y vezes. O caso-base é zero porque nenhuma parcela de x foi adicionada.

$$\text{mult}(x, y) = \text{cond}(y = 0, 0, \text{soma}(\text{mult}(x, p(y)), x))$$

$$\text{mult}(x, 0) = 0$$

$$\text{mult}(x, y) = \text{soma}(\text{mult}(x, p(y)), x) \text{ se } y \neq 0$$

Etapas	Expansão de $\text{mult}(4, 3)$	Valor
Base	$\text{mult}(4, 0)$	0
1	$\text{mult}(4, 1) = \text{soma}(\text{mult}(4, 0), 4) = \text{soma}(0, 4)$	4
2	$\text{mult}(4, 2) = \text{soma}(\text{mult}(4, 1), 4) = \text{soma}(4, 4)$	8
3	$\text{mult}(4, 3) = \text{soma}(\text{mult}(4, 2), 4) = \text{soma}(8, 4)$	12

Dependência: a multiplicação é primitiva recursiva porque usa uma quantidade de passos determinada por y e, em cada passo, chama a soma, que já foi construída como primitiva recursiva.

7. Fatorial construído com multiplicação

O fatorial de x multiplica todos os naturais positivos de 1 até x . O caso-base é $0!=1$. No passo recursivo, multiplicamos x pelo fatorial do predecessor.

$$\text{fat}(x) = \text{cond}(x = 0, 1, \text{mult}(x, \text{fat}(p(x))))$$

$$\text{fat}(0) = 1$$

$$\text{fat}(x) = \text{mult}(x, \text{fat}(p(x))) \text{ se } x \neq 0$$

Etapa	Expressão	Resultado
Base	$\text{fat}(0)$	1
1	$\text{fat}(1) = \text{mult}(1, \text{fat}(0)) = \text{mult}(1, 1)$	1
2	$\text{fat}(2) = \text{mult}(2, \text{fat}(1)) = \text{mult}(2, 1)$	2
3	$\text{fat}(3) = \text{mult}(3, \text{fat}(2)) = \text{mult}(3, 2)$	6
4	$\text{fat}(4) = \text{mult}(4, \text{fat}(3)) = \text{mult}(4, 6)$	24

Cadeia de construção: fatorial usa multiplicação; multiplicação usa soma; soma usa sucessor. Como as funções primitivas recursivas são fechadas por composição e recursão primitiva, toda a cadeia continua dentro da classe.